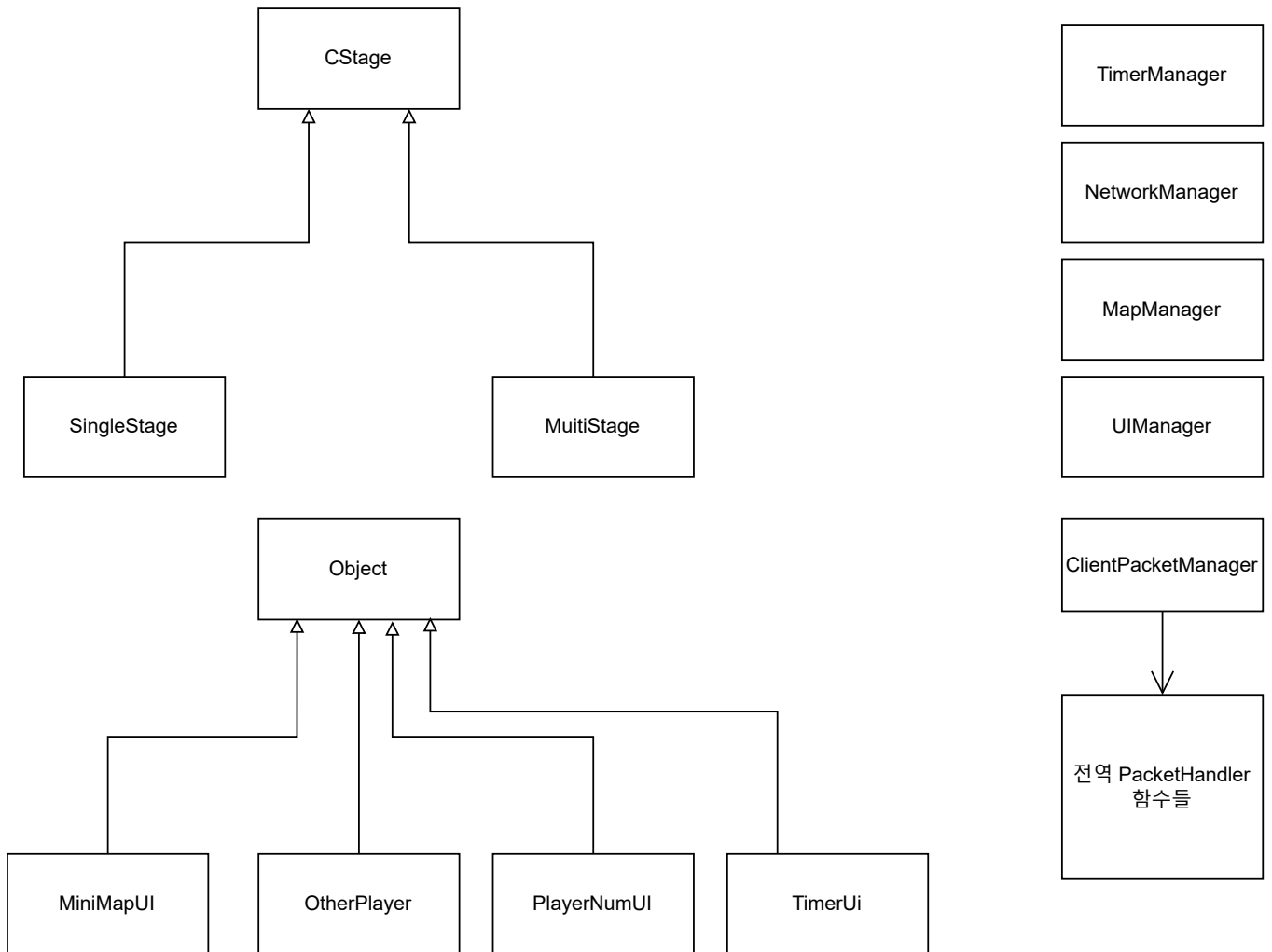
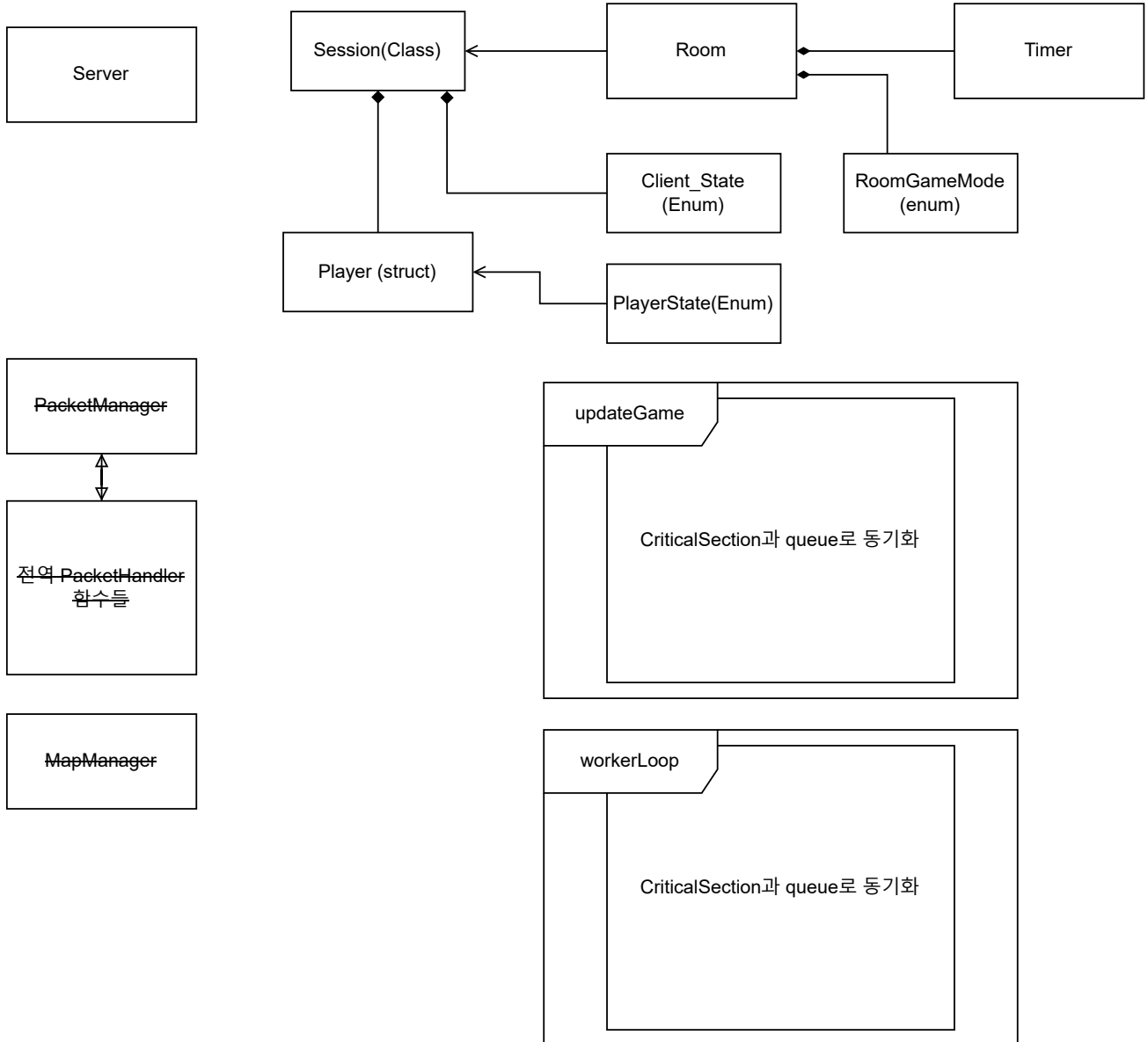


Client High Level Design



Server High Level Design



Client Low Level Design

Stage

```
Stage();  
virtual ~Stage();  
  
virtual void Initailizer() = 0;  
  
virtual void Update(float fTime) = 0;  
  
virtual void LateUpdate(float fTime) = 0;  
  
virtual void Render(HDC mDC) = 0;  
  
virtual void Free() = 0;
```

SingleStage : Stage

```
SingleStage();  
virtual ~SingleStage();  
  
virtual void Initailizer() override;  
  
virtual void Update(float fTime) override;  
  
virtual void LateUpdate(float fTime) override;  
  
virtual void Render(HDC mDC) override;  
  
virtual void Free() override;
```

MultiStage : Stage

```
MultiStage();  
virtual ~MultiStage();  
  
virtual void Initailizer() override;  
  
virtual void Update(float fTime) override;  
  
virtual void LateUpdate(float fTime) override;  
  
virtual void Render(HDC mDC) override;  
  
virtual void Free() override;
```

Object

```
Object();  
virtual ~Object();  
  
virtual void Initailizer() = 0;  
  
virtual void Update(float fTime) = 0;  
  
virtual void LateUpdate(float fTime) = 0;  
  
virtual void Render(HDC mDC) = 0;  
  
virtual void Free() = 0;  
  
SetPosRect();  
GetPosRect();  
  
SetCenterPos();  
GetCenterPos();  
  
ResetPosRect();  
  
SetRidius();  
  
GetRidius();  
  
SetDead();  
  
변수  
bool m_bDead{};  
RECT m_POSRECT{};  
POINT m_CenterPos{};  
float m_fRidius{};
```

MiniMapUI : Object

```
MiniMapUI();  
virtual ~MiniMapUI();  
  
virtual void Initailizer() override;  
  
virtual void Update(float fTime) override;  
  
virtual void LateUpdate(float fTime) override;  
  
virtual void Render(HDC mDC) override;  
  
virtual void Free() override;
```

PlayerNumUI : Object

```
PlayerNumUI();  
virtual ~PlayerNumUI();  
  
virtual void Initailizer() override;  
  
virtual void Update(float fTime) override;  
  
virtual void LateUpdate(float fTime) override;  
  
virtual void Render(HDC mDC) override;  
  
virtual void Free() override;
```

OtherPlayer : Object

```
OtherPlayer();  
virtual ~OtherPlayer();  
  
virtual void Initailizer() override;  
  
virtual void Update(float fTime) override;  
  
virtual void LateUpdate(float fTime) override;  
  
virtual void Render(HDC mDC) override;  
  
virtual void Free() override;
```

PlayerNumUI : Object

```
PlayerNumUI();  
virtual ~PlayerNumUI();  
  
virtual void Initailizer() override;  
  
virtual void Update(float fTime) override;  
  
virtual void LateUpdate(float fTime) override;  
  
virtual void Render(HDC mDC) override;  
  
virtual void Free() override;
```

<u>TimerManager</u>
GetInstance();
DestoryInstance();
GetDeltaTime();
GetFPS();
Reset();
Tick();
GetTotalTime();

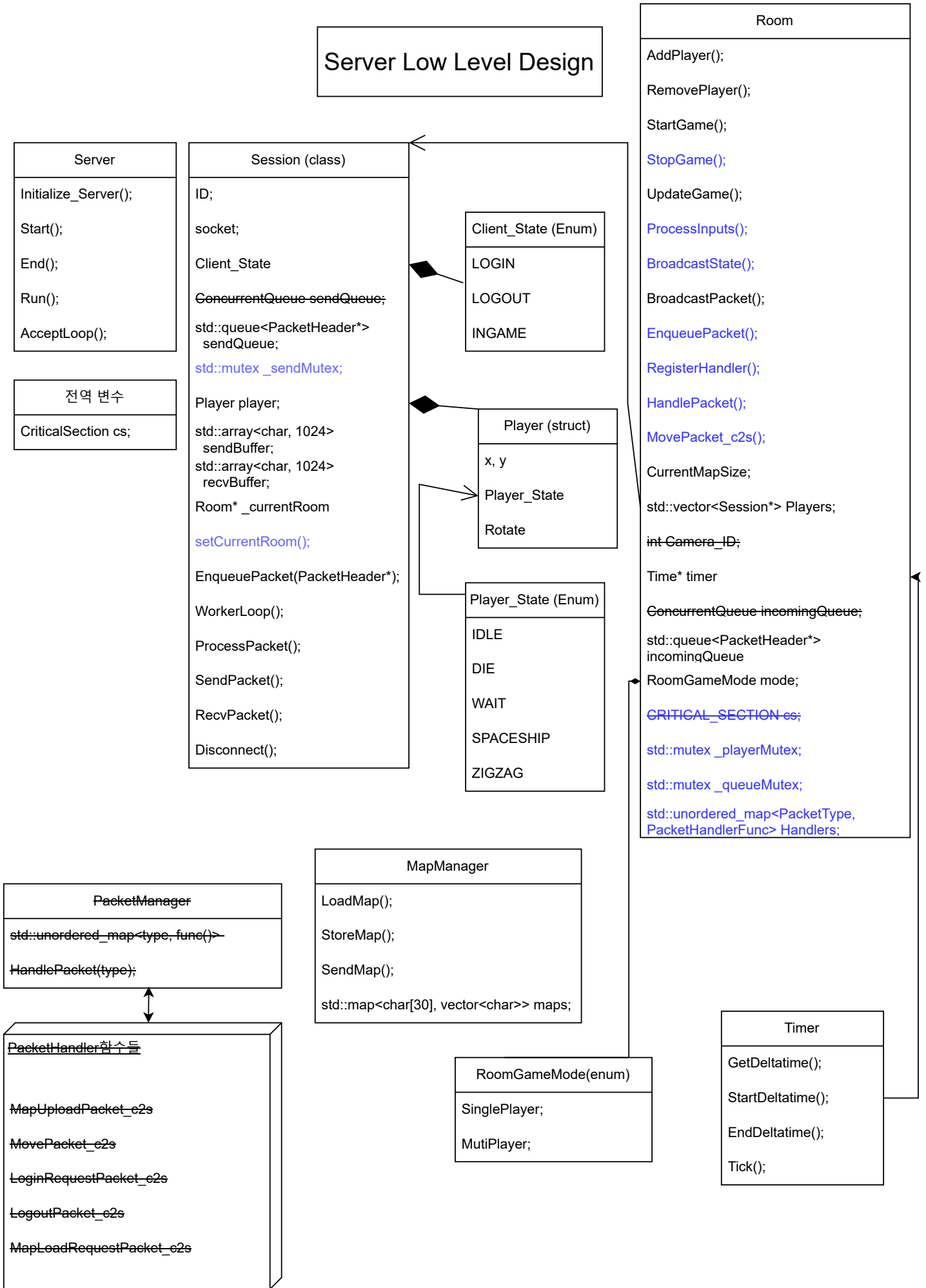
<u>MapManager</u>
GetInstance();
DestoryInstance();
MapShow();
MapLoad();
MapStage();

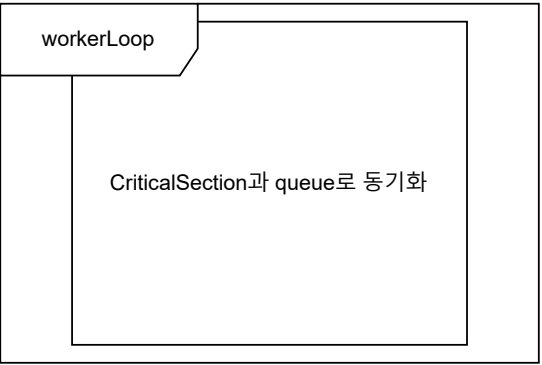
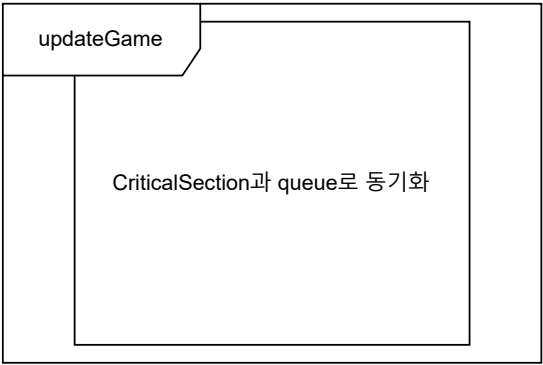
<u>NetworkManager</u>
GetInstance();
DestoryInstance();
initialize_Client();
sendPacket();
recvPacket();
UpdatePacket();
Shutdown();

<u>UIManager</u>
GetInstance();
DestoryInstance();
MiniMapUpdate();
MiniMapShow();
ScoreBoardShow();
TimerShow();
UserNumShow();

<u>전역 핸들러 함수</u>
MovePacket_s2c
LoginAcceptPacket_s2c
MapNamesPacket_s2c
MapLoadPacket_s2c
RoomEnterAcceptPacket_s2c
RoomStartPacket_s2c
MapUploadDonePacket_s2c
MapRoomEndPacket_s2c

Server Low Level Design





Packet

Client => Server

PacketManager
std::unordered_map<PacketType, func(> Handlers; RegisterHandler(type, func()); Handle(type,session);

Sever => Client

PacketManager
std::unordered_map<PacketType, func(> Handlers; RegisterHandler(type, func()); Handle(type);

MovePacket
long long size; PacketType type float x,y; int id float rotate int cameraID; Player_State player_state; float minMapPercentage;

MapNamesPacket
long long size; PacketType type std::array<char[30], 10> mapNames

MapLoadPacket
long long size; PacketType type char[30] mapNames vector<char> mapData;

LoginAcceptPacket
long long size; type int id

RoomEnterAcceptPacket
long long size; PacketType type
RoomStartPacket
long long size; PacketType type

MovePacket
long long size; PacketType type float x,y; float rotate Player_State player_state; float x,y;

S2C_AllPlayerMovePacket
long long size; PacketType type float x[3],y[3]; float roate[3] Player_State state[3]

LoginReqeustPacket
long long size; type

LogoutPacket
long long size; PacketType type

MapLoadRequestPacket
long long size; PacketType type char[30] mapName

RoomEnterPacket
long long size; PacketType type int roomID;

MapUploadDonePacket
long long size;
PacketType type

RoomEndPacket
long long size;
PacketType type
int roomID;
int winUserID
float ElapsedTime

MapUploadPacket
long long size;
PacketType type
char[30] mapName;
vector<char> mapData;

RoomLeavePacket
long long size;
PacketType type
int roomID;

PacketType(enum class)
MovePacket_s2c
MovePacket_c2s
LoginRequestPacket_c2s
LoginAcceptPacket_s2c
LogoutPacket_c2s
MapNamesPacket_s2c
MapLoadRequestPacket_c2s
MapLoadPacket_s2c
RoomEnterPacket_c2s
RoomEnterAcceptPacket_s2c
RoomStartPacket_s2c
MapUploadPacket_c2s
MapUploadDonePacket_s2c
MapRoomLeavePacket_c2s
MapRoomEndPacket_s2c